SWE015 HOMEWORK · USE CASE 2

Customer Support Ticket Router

Intent routing with a distilled, INT8-quantized BERT running on a CPU laptop

Student	Lina EL Fourgi
Student ID	2305025325
Repository	https://github.com/linafourgi-tech/SWE015-Customer-Support-Ticket-Router
Demo video	[Demo video URL]

Problem and goal

- Support teams receive free-text messages that must reach the right queue quickly.
- **Goal:** classify each new message into an intent, map it to a team, and send **uncertain or out-of-scope** messages to a human agent.
- **Constraint:** everything runs locally on a CPU-only laptop, so the model in the app must be small and fast.
- **Approach:** compare a four-model ladder, from classical NLP to a fine-tuned BERT, then compress BERT by **knowledge distillation followed by INT8 quantization**.

Success criteria (measured on one held-out test set)

Metric	Why it matters
Accuracy	Correct intent → correct team
OOS recall	Unsupported requests must reach a human
Size (MB)	Fits on a laptop
CPU latency	Interactive response

DEVELOPMENT

Dataset and preprocessing

CLINC150 `plus` (Larson et al., 2019)

Split	Messages	of which `oos`
Train	15,250	250
Validation (epoch selection)	3,100	100
Test (final evaluation only)	5,500	1,000

150 intents in 10 domains + 1 out-of-scope class = **151 labels**. Loaded from Hugging Face `clinc_oos`.

> **Scope:** CLINC150 is a **general assistant-intent** dataset (banking, travel, small talk, …). It is **not** an appliance-fault dataset. The product and model-code features are rule-based and were not evaluated.

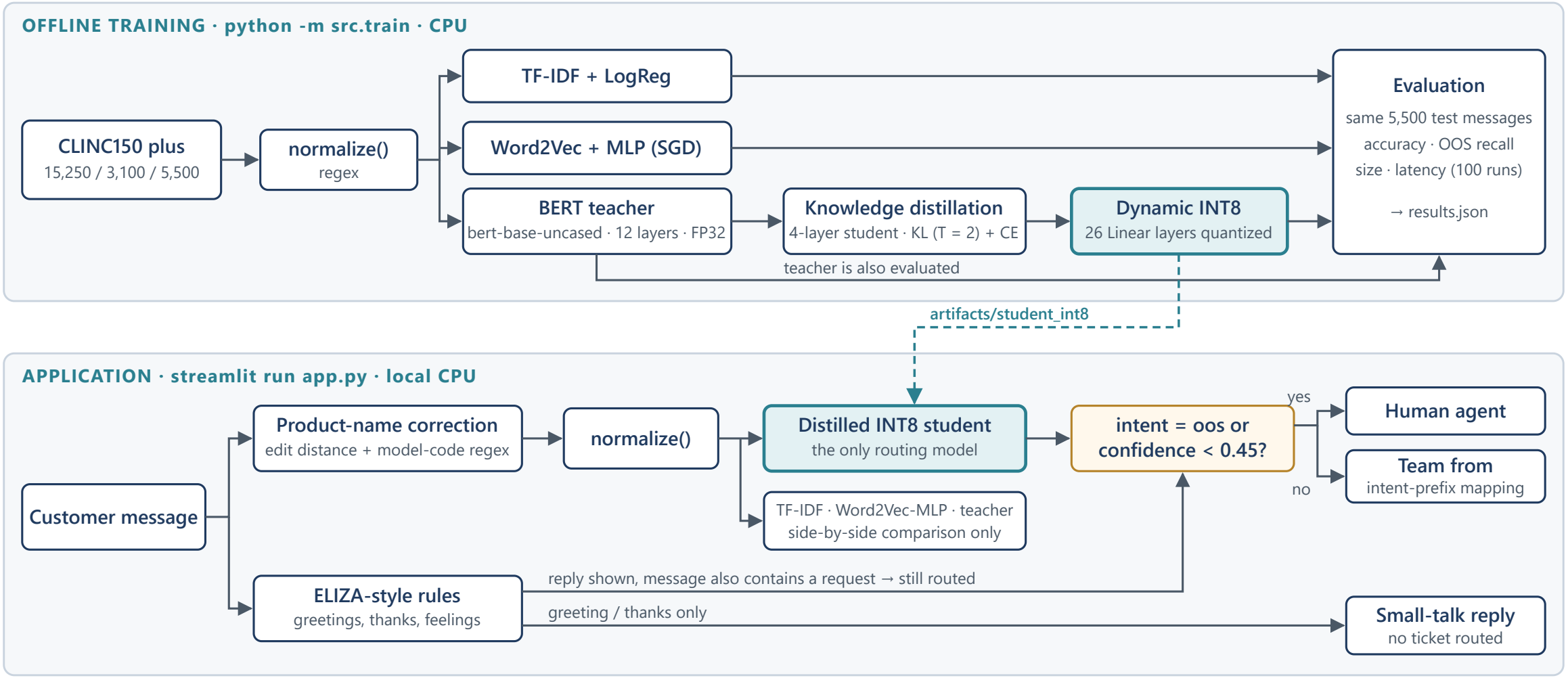
Regex normalization (`src/data.py`)

- Merge model codes before lowercasing: `WM-3500` → `wm3500`
- Lowercase; shorten repeated letters: `loooove` → `loove`
- Strip punctuation; collapse whitespace
- The same function is used in training, evaluation and the app

Bug found and fixed in the audit

The original code-merging regex also joined ordinary words to numbers (`send 100` → `send100`) in **646 of 23,850** texts (2.7%). The fixed rule only merges real model-code patterns.

System architecture



Technique map: the four required areas

Area	Topic	Technique in this project	Code
A	**Neural networks**	Skip-gram Word2Vec (100-d, trained on CLINC train texts), mean-pooled sentence vectors → MLP 100→128→151 (ReLU, dropout), **SGD + momentum, backpropagation**	`train.py: train_mlp` `models.py: MeanEmbeddingMLP`
B	**Classical NLP**	Regex normalization and model-code extraction · Levenshtein edit-distance product-name correction · ELIZA-style small-talk rules with pronoun reflection · **TF-IDF (1–2-grams) + logistic regression**	`data.py` `app.py: edit_distance, small_talk_reply`
C	**Transformers**	Fine-tuned `bert-base-uncased` sequence classifier (12 layers, hidden 768, 151 labels)	`train.py: train_models`
D	**Efficient models**	**Knowledge distillation** into a 4-layer BERT student, **then dynamic INT8 quantization** of the student; the app loads this model	`train.py: create_student_model, train_student` `models.py: quantize_int8`

Four-model ladder and training setup

#	Model	Training configuration
1	**TF-IDF + logistic regression**	1–2-grams, min_df 2, sublinear TF · LBFGS, C = 4
2	**Word2Vec + MLP**	Skip-gram 100-d, window 5, 8 epochs · MLP: SGD lr 0.05, momentum 0.9, 30 epochs, best epoch by validation loss
3	**BERT teacher (12 layers, FP32)**	`bert-base-uncased`, AdamW lr 2e-5, batch 32, **2 epochs**, max length 96 · validation loss 2.659 → 1.025
4	**Distilled student, 4 layers, INT8**	**Initialized from the teacher · 3 epochs of distillation (validation loss 0.705 → 0.471 → 0.398) · then INT8 quantization**

> Same seed (42), same official splits and the same normalization for every model. The test split is used only once, for the final benchmark.

> Training on the laptop CPU took about 1 h 45 min in total (teacher ≈ 54 min, distillation ≈ 51 min), estimated from log and artifact file timestamps.

Compression: distillation first, then quantization

Teacher
Fine-tuned BERT · 12 layers · 109.6 M parameters · FP32

→

Distilled student
4 layers · 52.9 M parameters · FP32

→

INT8 student (app)
26 Linear layers with int8 weights

Knowledge distillation (Hinton et al., 2015)

- Student = teacher config with **4 instead of 12** encoder layers, initialized from the teacher's embeddings, first 4 layers, pooler and classifier
- Teacher frozen (eval mode, no gradients); its logits are the soft targets
- Loss = $0.5 \cdot T^2 \cdot \mathrm{KL}(\text{teacher} \,\|\, \text{student}) + 0.5 \cdot \mathrm{CE}(\text{label})$, $T = 2$
- AdamW lr 3e-5; best validation epoch kept
- Parameters about halved (2.1×): the 23.4 M-parameter word-embedding table is not reduced

Dynamic INT8 quantization

- `torch.ao.quantization.quantize_dynamic` on the **student's** `nn.Linear` layers (qint8)
- Weights stored as int8; activations quantized at run time; CPU engine `onednn`
- Saved as state_dict + config; reloading was verified to give identical logits and 26 INT8 layers
- The app refuses to start if the loaded model is not smaller than the teacher or has no INT8 layers

Benchmark results (same 5,500 test messages)

Model	Accuracy	OOS recall	Size (MB)	Mean latency (ms) ± std
TF-IDF + logistic regression	0.823	0.470	14.3	1.33 ± 1.13
Word2Vec + MLP	0.636	0.258	2.4	0.21 ± 0.11
BERT teacher, 12 layers, FP32	0.843	0.420	419.0	56.76 ± 9.58
Distilled student, 4 layers, INT8 (app model)	**0.839**	**0.460**	**119.7**	**14.76 ± 6.95**

> **Teacher → INT8 student:** −0.4 accuracy points, 3.5× smaller on disk, 3.8× faster per message on CPU.

Supplementary note (not one of the four ladder models): the distilled student *before* quantization (FP32) reached accuracy 0.847, OOS recall 0.496, 202.7 MB, 18.93 ± 2.90 ms. So quantization cost about 0.8 accuracy points.

Hardware: Intel Core i7-1165G7 @ 2.80 GHz (4 cores / 8 threads), 11.8 GB RAM, CPU only (no GPU), Windows 11, Python 3.12.10, PyTorch 2.9.1+cpu (4 threads), Transformers 4.57.6.
Latency: batch size 1, raw string → normalize → tokenize → forward → softmax; 10 warm-ups + 100 timed runs.
Size: files needed to run the model (weights + config + tokenizer for the BERT models).
Source: `artifacts/results.json`

DEVELOPMENT

Live application and real failure examples

Observed in the app (INT8 student, default threshold 0.45)

Input	Prediction	Routing	Comment
"hi, my card was declined at the store"	`card_declined` 80.6%	Billing and payments	Greeting reply shown, request still routed
"My dishwaser WM-3500 is leaking water"	`w2` 9.2%	Human agent	Typo fixed → "dishwasher", code `WM3500`; topic not in CLINC → low confidence → fallback works
"how do i freeze my credit card"	`damaged_card` **66.4%**	General support	**Failure:** wrong but confident; above the threshold, so no human sees it

Test-set errors of the INT8 student (from `results.json`)

Message	True label	Predicted (confidence)	Observation
"how can i up my credit score"	`improve_credit_score`	`credit_score` (0.58)	Neighbouring intents confused
"vacation request please"	`pto_request`	`pto_request_status` (0.49)	Neighbouring intents confused
"repeat what the weather will be like"	`transfer`	`weather` (0.85)	The dataset label itself looks wrong

CONCLUSION

Conclusions

- **Distillation + INT8 kept almost all of the teacher's accuracy** (0.839 vs 0.843) with 3.5× less disk space and 3.8× lower CPU latency, so it is the right model for the CPU app.
- Quantization cost about 0.8 accuracy points (supplementary FP32 student: 0.847).
- The FP32 student slightly beat the 2-epoch teacher, most likely because it trained for 3 more epochs. This does **not** show that distillation beats the teacher in general.
- TF-IDF + logistic regression is a strong, cheap baseline (0.823 at about 1 ms). Word2Vec + MLP is clearly the weakest model (0.636).
- All four ladder models were trained and benchmarked locally on CPU with one reproducible script, and the app runs the compressed model end to end.

> **Main weakness:** out-of-scope recall is low for every model (0.26–0.47 across the four ladder models). Even the app model detects only 46% of out-of-scope test messages with plain argmax.

CONCLUSION

Limitations and future work

Limitations

- **Low OOS recall** (0.26–0.47). The benchmark uses plain argmax. The app's confidence threshold adds a human fallback, but its effect was not measured, and it cannot catch confident errors (see "freeze my credit card").
- The teacher trained for only 2 epochs on CPU; its validation loss was still falling.
- CLINC150 is not real support-ticket data. The team mapping and product/code rules are hand-written and not evaluated (e.g. "my machines are broken" is rewritten to "my machine are broken" and tagged as a washing machine).
- Latency varies with background load (TF-IDF: 0.73 ms in the training run vs 1.33 ms in the evaluation run).
- The embedding table stays FP32; `torch.ao.quantization` is deprecated (removal planned for PyTorch 2.10).

Future work

- Tune the OOS threshold on the validation split and report the resulting OOS recall and accuracy.
- Train the teacher for more epochs, then re-distill.
- Evaluate on real support tickets; train a dedicated appliance-fault classifier.
- Quantize the embeddings as well or migrate to `torchao`; try 2–3-layer students.

CONCLUSION

AI usage statement

Tool	How it was used
ChatGPT (OpenAI)	Used as a learning and research assistant to understand project requirements, explore technical concepts, plan the implementation, and get guidance when facing difficulties.
Claude Code (Anthropic)	Used as a coding assistant to support implementation, review code, troubleshoot errors, test functionality, and assist with project documentation.
My role	Project planning, dataset preparation and preprocessing, model implementation and training, performance evaluation, BERT fine-tuning, knowledge distillation, INT8 quantization, Streamlit application development, testing, documentation, and presentation.

> All benchmark numbers and test outcomes were produced by running the code locally on my laptop (`artifacts/results.json` , test logs). None of them were written or invented by an AI tool.